



https://urbagroupe.ma/events

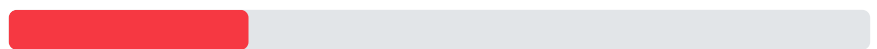
Report generated on Jun 15, 2026



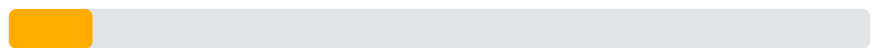
Average SEO score of top 100 sites: 74%

This website received an SEO score of **58 out of 100**, which is below the average score of **74**. However, there are **17 important issues** that need to be fixed to improve your website's ranking on search engines and enhance its overall performance.

17 Failed



6 Warnings



38 Passed



Issues to fix

HIGH

To address URL canonicalization issues, it is recommended to select a primary URL for your webpage and set up redirects from all other variations to the preferred one.

HIGH

To improve the website experience for your visitors, it is recommended to eliminate any render-blocking resources on this webpage.

HIGH

Users may abandon pages that take longer than 5 seconds to load, resulting in a potential loss of up to 50% of visitors. Faster loading pages can lead to increased traffic, better conversions, and higher sales.

HIGH

To ensure that Search Engines can accurately identify the topic of this webpage, it is important to include the most common keywords in the title tag, meta description, and heading tags.

HIGH

Build quality backlinks from websites relevant to your topic/niche to improve your search engine rankings.

HIGH

Add a sitemap file to help search engine crawlers index content more thoroughly and quickly.

HIGH

Add a robots.txt file to properly communicate with web crawlers and prevent unwanted access to sensitive content.



HIGH

Using images in a modern format can significantly reduce the file size and improve the loading speed of a webpage, providing a better user experience and potentially increasing engagement.

MEDIUM

Serve properly sized images to reduce page loading times and to improve user's experience.

MEDIUM

Avoid using distorted images, as they can have a negative impact on the user experience.

MEDIUM

By creating a custom 404 error page with helpful links and information, users are more likely to stay on the site and continue to explore.

MEDIUM

Consider adding cache headers for JavaScript resources to speed up the webpage for returning users.

LOW

Resolving errors identified by the Chrome DevTools Console can improve user experience.

LOW

While a canonical link tag can help in resolving duplicate content issues, it's important to verify that the specified URL is accurate, especially if the site's URL structure has been recently updated.

LOW

Using more than 20 HTTP requests on a webpage can negatively impact the loading time.

LOW

Consider adding the Strict-Transport-Security header to your webpage to ensure that web traffic is encrypted over HTTPS, mitigating the risk of man-in-the-middle attacks and other security threats.

Common SEO issues

7 Failed

2 Warnings

13 Passed



Meta Title Test

This webpage is using a title tag.

Text: URBA EVENTS | Agence Événementielle Premium au Maroc

Length: 52 characters



✔ Meta Description Test

This webpage is using a meta description tag.

Text: URBA EVENTS INTERNATIONAL — Agence leader d'événements premium au Maroc. Marathon de Marrakech, festivals culturels, séminaires d'entreprise et célébrations sur mesure.

Length: 168 characters

ℹ Google Search Results Preview Test

Desktop version

<https://urbagroupe.ma/events/>

URBA EVENTS | Agence Événementielle Premium au Maroc

URBA EVENTS INTERNATIONAL — Agence leader d'événements premium au Maroc. Marathon de Marrakech, festivals culturels, séminaires d'entreprise et célébration...

Mobile version

<https://urbagroupe.ma/events/>

URBA EVENTS | Agence Événementielle Premium au...

URBA EVENTS INTERNATIONAL — Agence leader d'événements premium au Maroc. Marathon de Marrakech, festivals culturels,...



✓ Social Media Meta Tags Test

This webpage is using social media meta tags.

Open Graph Meta Tags

og:type	website
og:url	https://urbaevents.com/
og:title	URBA EVENTS Agence Événementielle Premium au Maroc
og:description	Créateur d'événements sportifs et culturels inoubliables au Maroc. Marathon de Marrakech, festivals, séminaires et célébrations sur mesure.
og:image	https://urbaevents.com/assets/imgs/og-image.png
og:image:width	1200
og:image:height	630
og:image:type	image/png
og:locale	fr_FR
og:site_name	URBA EVENTS INTERNATIONAL

Twitter Card Meta Tags

twitter:card	summary_large_image
twitter:title	URBA EVENTS Agence Événementielle Premium au Maroc
twitter:description	Créateur d'événements sportifs et culturels inoubliables au Maroc.
twitter:image	https://urbaevents.com/assets/imgs/og-image.png

i Most Common Keywords Test

There is likely no optimal keyword density (search engine algorithms have evolved beyond keyword density metrics as a significant ranking factor). It can be useful, however, to note which keywords appear most often on your page and if they reflect the intended topic of your page. More importantly, the keywords on your page should appear within natural sounding and grammatically correct copy.

11 marrakech 9 notre 8 édition 8 cookies 6 votre



✖ Keywords Usage Test

The most common keywords of this webpage are not distributed across the important HTML tags! Primary keywords should appear in title tag, meta description and heading tags to help Search Engines to properly identify the topic of this webpage.

Keyword	Title tag	Meta description	Headings
marrakech	✖	✓	✖
notre	✖	✖	✖
édition	✖	✖	✖
cookies	✖	✖	✖
votre	✖	✖	✖

i Keywords Cloud Test





Related Keywords Test

This URL is currently ranked in the top 20 organic Google listings for the search terms below:

- [2026 ⵓⵎⵎⵓⵔ ⵎⵎⵓⵔⵎⵎⵓⵔ ⵎⵎ ⵎⵎⵎⵎⵎⵎⵎⵎ](#)
- [marathon marrakech](#)
- [2026 ⵓⵎⵎⵓⵔ ⵎⵎⵓⵔⵎⵎⵓⵔ ⵎⵎⵎⵎ](#)
- [ⵎⵎⵎⵎⵎⵎ ⵎⵎⵎⵎⵎⵎⵎⵎ](#)
- [marathon marrakech 2025](#)
- [2026 ⵓⵎⵎⵓⵔ ⵎⵎⵓⵔⵎⵎⵓⵔ](#)
- [marathon maroc 2026](#)
- [semi-marathon marrakech 2026](#)

Competitor Domains Test

Identifying competitor domains allows you to find domains that are competing with your website for search engine rankings on similar keywords. These domains typically target the same audience and have overlapping goals in terms of organic traffic.



! Heading Tags Test

This webpage contains too many H2 tags! H2 tags should re-inforce the related content of your page to search engines - too many tags may make the topic less clear, or look like spam tactics. Consider using less than 10 H2 tags.

H1 tags

Créateur d'Événements Inoubliables

Transformez Vos Idées En Réalité

Des Moments Inoubliables

H2 tags

Make it Unique

L'Exceptionnel en 3D

Derniers Travaux

Nos Services

Événements Corporatifs

Événements Privés

Événements Sportifs

Festivals & Culture

Production Audiovisuelle

Aménagement & Logistique

Nos Articles



Robots.txt Test

This website lacks a "robots.txt" file. This file can protect private content from appearing online, save bandwidth, and lower load time on your server. A missing "robots.txt" file also generates additional errors in your apache log whenever robots request one. Read more about the [robots.txt file](#), and how to create one for your site.



How to pass this test?

The robots.txt file is the first resource most crawlers request and tells them which paths on your site they may or may not access. Fixing this issue means publishing a valid robots.txt at the site root so search engines, training crawlers, and answer-engine bots all have an explicit policy to follow. Without it, every crawler defaults to its own behavior, which can over-expose private sections or waste crawl budget on low-value URLs.

Example

```
User-agent: *  
Disallow: /admin/  
Disallow: /cart  
Allow: /
```

Sitemap: <https://example.com/sitemap.xml>

Where to make the change

- **Raw HTML or static site:** create a plain text file named `robots.txt` and upload it to the document root so it is served at <https://example.com/robots.txt>.
- **WordPress:** WordPress generates a virtual robots.txt by default. Override it with a physical file in the site root, or use your SEO plugin's robots.txt editor.
- **Shopify:** Shopify auto-generates robots.txt and exposes a `robots.txt.liquid` template you can edit from the theme code editor to add or remove rules.
- **Wix or Squarespace:** both platforms generate robots.txt automatically and offer limited overrides through the SEO settings panel.
- **Headless or framework sites:** add `robots.txt` to the project's public or static directory so it ships with the build.

Common causes and how to resolve them

- **File is missing entirely:** create one. Even an empty file with `User-agent: *` and no rules is preferable to a 404.
- **File returns a 5xx error:** some crawlers stop crawling the entire site when robots.txt is unreachable. Investigate server or CDN issues that block the path.
- **Accidentally blocking the whole site:** a stray `Disallow: /` under `User-agent: *` hides every URL from search. Audit the file before deploying changes from staging.
- **Wrong location:** robots.txt must sit at the site root, not in a subdirectory. Crawlers do not look elsewhere.

Best practices

- **Reference the sitemap:** add a `Sitemap:` line so any crawler that fetches robots.txt also discovers your XML sitemap.
- **Keep it simple:** a short, well-commented file is easier to audit than a sprawling one full of legacy rules. Remove directives whose original purpose is no longer relevant.
- **Do not rely on robots.txt for security:** the file is public, so listing private paths in it advertises them. Block sensitive paths with authentication, not just `Disallow`.



- **Manage AI crawlers explicitly:** if you want to allow or block bots like `GPTBot`, `ClaudeBot`, or `PerplexityBot`, add named `User-agent` sections for each. Defaulting to allow keeps your content eligible for citation in AI answer engines.



Sitemap Test

This website lacks a sitemap file! Sitemaps can help robots index your content more thoroughly and quickly. Read more on Google's guidelines for [implementing the sitemap protocol](#).



How to pass this test?

An XML sitemap gives search engines and AI crawlers an explicit, machine-readable list of the URLs you want indexed, along with metadata such as last-modified dates. Fixing this issue means generating a valid sitemap, publishing it at a stable URL, and pointing crawlers to it via robots.txt and Search Console. This is especially important for large sites, sites with weak internal linking, or sites where new content needs to be discovered quickly.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://example.com/</loc>
    <lastmod>2026-04-15</lastmod>
  </url>
  <url>
    <loc>https://example.com/blog/getting-started</loc>
    <lastmod>2026-03-22</lastmod>
  </url>
</urlset>
```

Where to make the change

- **Raw HTML or static site:** generate the XML at build time and publish it at `/sitemap.xml`. Many static-site generators have a built-in sitemap option.
- **WordPress:** a dedicated SEO plugin will generate and update the sitemap automatically, typically at `/sitemap_index.xml`.
- **Shopify:** Shopify generates `/sitemap.xml` automatically and updates it as products and collections change.
- **Wix or Squarespace:** both platforms generate the sitemap automatically and serve it at the standard root path.
- **Headless or framework sites:** use the framework's built-in sitemap module (for example `next-sitemap` for Next.js or the `@astrojs/sitemap` integration) so the file is regenerated on every build.

Common causes and how to resolve them

- **No sitemap published:** generate one and publish it at the site root. Reference it from robots.txt with a `Sitemap:` line.
- **Sitemap lists URLs that 404 or redirect:** stale entries waste crawl budget and trigger Search Console warnings. Regenerate the sitemap from current published URLs only.
- **Mixed canonical and non-canonical URLs:** the sitemap should list canonical URLs only. Listing both `http` and `https`, or trailing-slash variants, splits crawl signals.
- **Sitemap exceeds size limits:** a single sitemap may contain at most 50,000 URLs and 50 MB uncompressed. Split larger sites into multiple sitemaps and reference them from a sitemap index file.

Best practices

- **Submit to Search Console:** add the sitemap URL in Google Search Console (and Bing Webmaster Tools) so the platform tracks coverage and surfaces errors.



- **Keep `lastmod` accurate:** Google uses `lastmod` as a hint when prioritizing crawls. Update it whenever the page content meaningfully changes; do not bump it on every build.
- **Exclude `noindex` and `disallowed` URLs:** the sitemap should describe what you want indexed. Listing pages that are blocked from crawling sends mixed signals.
- **Use sitemap index files for multi-sitemap setups:** a sitemap index aggregates child sitemaps (for example one per content type) and is the single URL you submit to search engines.



Image Alt Test

This webpage is using "img" tags with empty or missing "alt" attribute!



How to pass this test?

Alt attributes provide a text alternative for images, used by screen readers when a user cannot see the image and by search engines and multimodal AI models to understand what the image depicts. Fixing this issue means writing concise, descriptive alt text for every meaningful image on the page, and using empty alt for purely decorative images so assistive technology can skip them. The same alt text helps multimodal LLMs and image search tools surface and cite your visuals.

Example

```

```

```

```

Where to make the change

- **Raw HTML:** add the `alt` attribute to every `<img>` tag. For decorative images, use `alt=""` so screen readers skip them.
- **WordPress:** the media library has an Alt text field on every uploaded image. Fill it once and the value is reused everywhere the image is inserted.
- **Shopify:** in the product or page editor, click an image and use the Edit alt text option. Theme code that renders product images should pull `image.alt`.
- **Wix or Squarespace:** click the image in the editor and use the Alt text or Image description field in the side panel.
- **Headless or framework sites:** require an `alt` prop on every image component and lint for missing values during build.

Common causes and how to resolve them

- **Attribute missing:** add `alt=""` at minimum so the page validates. Replace with descriptive text on any image that conveys information.
- **Alt text is the filename:** "IMG_4837.jpg" tells the user nothing. Replace with a description of what is in the image.
- **Alt text duplicates the visible caption:** screen readers will read both, doubling the user's listening time. Differentiate the alt from the caption or empty the alt.
- **Decorative images get descriptive alt text:** background patterns and dividers should use `alt=""`, otherwise screen readers narrate noise.
- **Bulk imports left alt blank:** a one-off audit of the media library or a CMS-side script can fill missing alt values from product names, captions, or asset descriptions.

Best practices

- **Describe the function, not just the appearance:** for a chart, describe what the chart shows. For a button image, describe what the button does.
- **Keep it concise:** aim for under 125 characters. Screen readers do not pause mid-sentence, so long alt text is exhausting to listen to.
- **Skip "image of" or "picture of":** the screen reader already announces it as an image.



- **Include the focus keyword when natural:** alt text is a minor on-page signal, but stuffing keywords in unnatural alt text harms both accessibility and ranking.



Responsive Image Test

Not all images in this webpage are properly sized! This webpage is serving images that are larger than needed for the size of the user's viewport.



How to pass this test?

This test fails when one or more images on the page are served at dimensions significantly larger than the size at which the browser actually renders them. Fixing this issue means resizing the source files (or serving responsive variants) so the bytes downloaded match the pixels actually displayed. The result is faster page loads, lower data usage, and better Core Web Vitals scores.

Example

```

```

Where to make the change

- **Raw HTML:** use `srcset` and `sizes` to let the browser pick the right variant for the user's viewport. Always include intrinsic `width` and `height` attributes.
- **WordPress:** WordPress generates multiple image sizes automatically. Use the `the_post_thumbnail()` helper or the block editor's image block, both of which emit responsive `srcset` markup.
- **Shopify:** use the `image_url` Liquid filter with size parameters (for example `{{ image | image_url: width: 800 }}`) and pair with `image_tag` to emit a responsive image.
- **Wix or Squarespace:** both platforms generate responsive variants automatically; this issue is typically caused by uploading a deliberately oversized image.
- **Headless or framework sites:** use the framework's image component (for example `next/image` or `astro:assets`), which generates responsive variants and lazy-loads off-screen images.

Common causes and how to resolve them

- **Original camera or stock-photo file uploaded as-is:** a 6000-pixel-wide JPEG rendered in a 400-pixel column wastes most of the bytes. Resize the source or generate responsive variants.
- **No `srcset`:** the browser has no choice but to use the single source. Add multiple variants so mobile users get smaller files.
- **Hero images sized for ultra-wide displays:** if your largest layout is 1600 pixels, do not ship a 4000-pixel original.
- **Background images set in CSS without responsive sources:** use the `image-set()` CSS function or media queries to swap in different image files at different breakpoints.

Best practices

- **Generate variants at common widths:** 400, 800, 1200, and 1600 pixels covers most viewports cleanly.
- **Always set width and height:** intrinsic dimensions let the browser reserve space and avoid layout shift, helping CLS.
- **Lazy-load off-screen images:** add `loading="lazy"` to images below the fold so the browser only downloads them when needed.



- **Pair with modern formats:** resizing alone is good; resizing and serving WebP or AVIF compounds the savings.



✖ Image Aspect Ratio Test

Not all image display dimensions match the natural aspect ratio! Fix aspect ratio issues to avoid distorted images on this website!

How to pass this test?

This test fails when one or more images are rendered at an aspect ratio meaningfully different from the source file's native ratio, producing stretched or squashed visuals. Fixing this issue means either using `width` and `height` attributes that match the file's real dimensions or replacing the source with a correctly proportioned crop.

Example

```
<!-- Source file is 1600 x 900 (16:9). Set matching attributes: -->

```

Where to make the change

- **Raw HTML and CSS:** set the `width` and `height` attributes to the source file's native dimensions. Use CSS `height: auto` when scaling responsively so the browser preserves the ratio.
- **WordPress:** when inserting an image via the block editor, choose the image's intended size rather than overriding dimensions in custom CSS.
- **Shopify, Wix, or Squarespace:** use the platform's built-in image cropper so the displayed crop matches the source file you upload.
- **Headless or framework sites:** components like `next/image` require both `width` and `height`, which prevents distortion when the layout scales.

Common causes and how to resolve them

- **Hard-coded width and height that do not match the file:** recompute them from the source dimensions or remove them and rely on the file's intrinsic ratio.
- **Container has a fixed height that crops the image:** use CSS `object-fit: cover` if cropping is intentional, or change the container so the image keeps its natural ratio.
- **Wrong source file uploaded:** swap in a correctly proportioned crop rather than stretching the wrong file in CSS.

Best practices

- **Always set both `width` and `height`:** the browser uses these to reserve space and avoid layout shift before the image loads, which helps CLS.
- **Use `object-fit` when cropping is intentional:** `cover` fills the container without distortion; `contain` letterboxes to preserve the full image.
- **Standardize on a few aspect ratios:** picking one or two ratios per template makes it easier to crop assets consistently and avoid one-off distortions.



✔ Deprecated HTML Tags Test

This webpage does not use HTML deprecated tags.

✔ Google Analytics Test

This webpage is using Google Analytics.

✔ Favicon Test

ⓘ This website appears to have a favicon.

✖ Backlinks Test

We didn't find any backlinks for this domain! Quality backlinks from websites relevant to your topic/niche are critical for strong search engine rankings.

✔ JS Error Test

There are no severe JavaScript errors on this webpage.



✖ Console Errors Test

This webpage has some errors caught by the Chrome DevTools Console!

How to pass this test?

Console errors include any unhandled JavaScript exceptions, network request failures, security warnings, or deprecation notices the browser emits while loading the page. Fixing this issue means resolving the underlying problem each error reports rather than suppressing the message. Even errors that do not visibly break the page can degrade performance, leak data, or trip Core Web Vitals.

Where to make the change

- **Open DevTools, Console tab:** expand each entry to see the file, line, and stack trace. Right-click an entry to filter, save, or jump to the source.
- **Application code:** patch the offending script (see js-errors). For network failures, fix the failing endpoint or remove the stale request.
- **Third-party tags:** if the error originates in an analytics, ad, or widget script, update the snippet, switch to the vendor's current loader, or remove unused tags.

Common causes and how to resolve them

- **JavaScript runtime errors:** see the JS Error Test fix; the same approach applies.
- **Failed network requests (404 or 5xx):** remove the dead request or fix the endpoint. Stale tracking pixels and removed assets are common culprits.
- **Mixed content warnings:** the page is HTTPS but loads an HTTP resource. Update the resource URL to `https://`.
- **CORS or CSP violations:** tighten or expand the policy as appropriate. CSP errors usually require allowlisting the offending origin in the response header.
- **Deprecation warnings:** update the call site to use the recommended API. These do not break the page today but will when the deprecation cycle completes.

Best practices

- **Treat the console as a build artifact:** a clean console is a sign of a healthy page. Aim for zero errors and zero warnings on production routes.
- **Use error monitoring in production:** real users hit edge cases your local environment will never reproduce. A monitoring service surfaces them with frequency data.
- **Defer or remove unused third-party scripts:** every removed script is one fewer source of console noise.

✔ Charset Declaration Test

This webpage has a character encoding declaration.

```
meta charset="utf-8"
```



Speed optimizations

5 Failed

3 Warnings

12 Passed

✓ HTML Page Size Test

The size of this webpage's HTML is **10.6 Kb** and is under the average webpage's HTML size of **33 Kb**. Faster loading websites result in a better user experience, higher conversion rates, and generally better search engine rankings.

✓ DOM Size Test

The Document Object Model (DOM) of this webpage has **722 nodes** which is less than the recommended value of 1,500 nodes.

✓ HTML Compression/GZIP Test

This webpage is successfully compressed using **br compression** on your code. The HTML code is compressed from **51.71 Kb** to **10.6 Kb** (**79% size savings**). This helps ensure a faster loading webpage and improved user experience.



✖ Site Loading Speed Test

The loading time of this webpage (measured from N. Virginia, US) is around **7.86 seconds** and is greater than the average loading speed which is **5 seconds**!

How to pass this test?

This test fails when the overall time to load the page exceeds an acceptable threshold. Page speed is a confirmed Google ranking factor and a strong driver of bounce rate; slower pages lose a meaningful share of users before they ever see the content. Fixing this issue usually requires multiple coordinated changes across server, assets, and rendering rather than a single tweak.

Where to make the change

- **Server and hosting:** upgrade the host plan, enable a CDN, and turn on caching. A slow server is the floor for every other metric.
- **Front-end assets:** compress and resize images, minify CSS and JavaScript, defer non-critical scripts, and preload critical resources.
- **Application code:** render server-side or pre-render static content where possible; lazy-load heavy components below the fold.

Common causes and how to resolve them

- **Slow Time to First Byte:** investigate server response time, database queries, and third-party API calls. Cache expensive operations.
- **Heavy unoptimized images:** resize to displayed dimensions, serve modern formats (WebP, AVIF), and lazy-load images below the fold.
- **Render-blocking CSS and JavaScript:** inline the critical CSS for above-the-fold content and defer non-critical scripts with `defer` or `async`.
- **Too many third-party scripts:** audit each tag (analytics, ads, chat, A/B test) and remove what is not actively delivering value. Load remaining tags after main content.
- **No CDN:** serving assets from a single origin server adds latency for distant users. A CDN moves files to edge nodes near the visitor.

Best practices

- **Target a Lighthouse performance score above 90 on mobile:** mobile is the indexed view and the most demanding rendering environment.
- **Set a performance budget:** define maximum JavaScript bundle size, total page weight, and Core Web Vitals targets, then enforce them in CI.
- **Measure with real-user data:** Chrome User Experience Report (CrUX) and tools such as PageSpeed Insights surface field data alongside lab data for a complete picture.
- **Optimize the critical rendering path:** minimize the number of resources required to render above-the-fold content; everything else can wait.



✔ JS Execution Time Test

The JavaScript code used by this webpage is executed in less than **2 seconds**.



✖ Page Objects Test

This webpage is using more than 20 http requests, which can slow down page loading and negatively impact user experience!

Content size by content type

Content type	Percent	Size
Image	95.1 %	26.87 Mb
Font	2.8 %	799.62 Kb
Javascript	1.6 %	463.97 Kb
Css	0.6 %	159.89 Kb
Html	0.0 %	8.30 Kb
Other	0.0 %	0 B
TOTAL	100%	28.27 Mb

Requests by content type

Content type	Percent	Requests
Image	41.2 %	35
Css	22.4 %	19
Javascript	16.5 %	14
Font	10.6 %	9
Html	9.4 %	8
Other	0.0 %	0
TOTAL	100%	85

Content size by domain

Domain	Percent	Size
urbagroupe.ma	97.3 %	27.50 Mb
uithemez.com	1.7 %	502.92 Kb
googletagmanager.com	0.6 %	160.66 Kb



fonts.gstatic.com	0.2 %	69.29 Kb
cdn.jsdelivr.net	0.2 %	45.00 Kb
fonts.googleapis.com	0.0 %	1.55 Kb
TOTAL	100%	28.27 Mb

Requests by domain

Domain	Percent	Requests
urbagroupe.ma	56.5 %	48
uithemex.com	24.7 %	21
googletagmanager.com	9.4 %	8
fonts.gstatic.com	5.9 %	5
cdn.jsdelivr.net	2.4 %	2
fonts.googleapis.com	1.2 %	1
TOTAL	100%	85



! CDN Usage Test

This webpage is not serving all resources (images, javascript and css) from CDNs!

How to pass this test?

A Content Delivery Network serves static assets from edge locations close to the user, reducing latency, improving cache hit rates, and offloading traffic from the origin server. Fixing this issue means routing your images, JavaScript, CSS, and fonts through a CDN so visitors anywhere in the world get fast, consistent delivery.

Where to make the change

- **Cloud platforms:** Cloudflare, Fastly, AWS CloudFront, Bunny.net, and similar providers offer free or low-cost CDN tiers. Sign up, point your domain through the CDN, and enable caching for static asset paths.
- **WordPress:** use a CDN integration plugin or sign up directly with Cloudflare and update the site's nameservers.
- **Shopify:** Shopify serves storefront assets through its own CDN automatically; the failure usually points at custom assets uploaded outside the platform.
- **Headless or framework sites:** deploying to Vercel, Netlify, or Cloudflare Pages puts the entire site behind a global CDN by default.

Common causes and how to resolve them

- **No CDN at all:** add one. Even a free Cloudflare account in front of an existing host produces measurable speed gains.
- **CDN configured but bypassed for some assets:** ensure all `img`, `script`, `link`, and `font` URLs use the CDN hostname rather than the origin.
- **Cache headers prevent caching:** long-lived assets need `cache-control` headers like `public, max-age=31536000, immutable`. Without them, the CDN cannot cache effectively.
- **Origin-only third-party scripts:** many vendors already serve their scripts via their own CDN. Use the URL the vendor provides rather than self-hosting.

Best practices

- **Use a versioned asset path:** filenames like `app.a3f9.css` let you set far-future cache headers without ever serving stale files. Update the filename when content changes.
- **Enable Brotli at the edge:** most CDNs can compress responses on the fly, including text formats.
- **Pre-warm caches for new launches:** after a major release, request key URLs through every region (or use the CDN's purge-and-prefetch tools) so users do not hit cold caches.



Modern Image Format Test

This webpage is not serving images in a modern format! Image formats like [JPEG 2000](#), [JPEG XR](#), and [WebP](#) often provide better compression than PNG or JPEG, which means faster downloads and less data consumption.



How to pass this test?

This test fails when images are served in legacy formats such as JPEG and PNG instead of modern alternatives like WebP or AVIF. Modern formats compress significantly better at equivalent visual quality, which means smaller transfers, faster page loads, and lower data usage for visitors on metered connections. Fixing this issue means generating modern variants and serving them to browsers that support them.

Example

```
<picture>
  <source srcset="/images/hero.avif" type="image/avif">
  <source srcset="/images/hero.webp" type="image/webp">
  
</picture>
```

Where to make the change

- **Raw HTML:** use the `<picture>` element with multiple `<source>` entries so the browser picks the best supported format.
- **WordPress:** WordPress 6.x and later supports WebP natively. Use a media optimization plugin to convert existing JPEGs and PNGs and serve modern formats automatically.
- **Shopify:** Shopify's image CDN can serve WebP automatically; using the `image_url` filter with `format` parameter or the `image_tag` helper handles negotiation.
- **Wix or Squarespace:** both platforms convert and serve WebP automatically when supported by the visitor's browser.
- **Headless or framework sites:** components like `next/image` and `astro:assets` generate AVIF and WebP variants at build time and serve the best format per request.

Common causes and how to resolve them

- **Only legacy formats published:** add a build step or image CDN that generates WebP and AVIF alongside the original.
- **Single-format `<img>` tag:** wrap the image in `<picture>` with `<source>` entries for AVIF and WebP, falling back to the JPEG.
- **Image CDN not negotiating format:** enable automatic format conversion or pass a `format=auto` query parameter (depends on provider).
- **SVG used where a raster image would be smaller (or vice versa):** illustrations and icons should be SVG; photos should be raster in a modern format. Mixing them up wastes bytes.

Best practices

- **AVIF first, WebP fallback, JPEG last:** AVIF compresses best, WebP is widely supported, JPEG remains the universal fallback.
- **Match quality to use case:** hero images can use higher quality settings; thumbnails can drop quality without visible loss.
- **Use an image CDN:** services like Cloudflare Images, Cloudinary, or Imgix handle resizing, format conversion, and caching in one step.



✓ Image Metadata Test

This webpage is not using images with large metadata.

✓ Image Caching Test

This website is using cache headers for images and the browsers will display these images from the cache.

✗ JavaScript Caching Test

This webpage is not using cache headers for JavaScript resources! Setting cache headers can help to speed up the webpage for returning users.

How to pass this test?

This test fails when external JavaScript files are served without caching headers that allow browsers to store and reuse them. Cached scripts skip the network on repeat visits, reducing load time and bandwidth for returning users. Fixing this issue is a server or CDN configuration change.

Where to make the change

- **Server configuration:** add `Cache-Control: public, max-age=31536000, immutable` to JavaScript responses, paired with versioned filenames.
- **CDN configuration:** set a default cache rule for `*.js` assets at the edge.
- **Build pipeline:** emit hashed filenames (for example `app.7f3c1e.js`) so cache lifetimes can be long without risking stale code.
- **WordPress and plugins:** caching and CDN plugins handle JS cache headers automatically.

Common causes and how to resolve them

- **No `Cache-Control` header:** add one. Without it, browsers revalidate on every page load.
- **Short `max-age`:** bump to a year for versioned files. Use a shorter lifetime only for unversioned scripts that may change in place.
- **`no-cache` or `no-store` on static JS:** remove these directives; they belong on dynamic responses, not static assets.
- **Cookies on script requests:** serve scripts from a cookie-free subdomain or a CDN to enable shared caching.

Best practices

- **Versioned filenames:** hashed file names plus `immutable` caching is the gold standard for cache effectiveness.
- **Self-host critical scripts:** hosting third-party scripts yourself lets you control cache headers and avoid vendor outages.
- **Audit with DevTools:** the Network panel's "Disk cache" column shows whether each script is genuinely served from cache on repeat visits.



✓ CSS Caching Test

This webpage is using cache headers for all CSS resources.

✓ JavaScript Minification Test

All JavaScript files used by this webpage are minified.

✓ CSS Minification Test

All CSS resources used by this webpage are minified.



Render Blocking Resources Test

This webpage is using render blocking resources! Eliminating render-blocking resources can help this webpage to load significantly faster and will improve the website experience for your visitors.



How to pass this test?

This test fails when JavaScript or CSS resources block the browser from rendering visible content. Each render-blocking file delays the first paint, and on slower networks the cumulative impact can be severe. Fixing this issue means inlining the truly critical CSS, deferring or asynchronously loading non-essential scripts, and removing unused code.

Example

```
<head>
<!-- Inline only above-the-fold critical CSS -->
<style>/* critical styles here */</style>

<!-- Load full stylesheet asynchronously -->
<link rel="preload" href="/styles/main.css" as="style"
      onload="this.onload=null;this.rel='stylesheet'">

<!-- Defer non-critical scripts -->
<script src="/scripts/app.js" defer></script>
</head>
```

Where to make the change

- **Application code or templates:** identify the CSS and JavaScript needed for the initial render and load only those synchronously. Defer the rest.
- **Build pipeline:** use a critical CSS extractor (such as `critical` or `penthouse`) to generate the above-the-fold styles automatically.
- **WordPress:** performance plugins offer "delay JS execution" and "load CSS asynchronously" options that handle most of this without code changes.
- **Headless or framework sites:** use the framework's preload and code-splitting hints to control what blocks rendering.

Common causes and how to resolve them

- **Single large stylesheet blocking render:** inline critical CSS for the first viewport, load the full stylesheet asynchronously.
- **Synchronous third-party scripts in the head:** add `defer` or `async`, or move them to the end of the body.
- **Web fonts blocking text rendering:** add `font-display: swap` in `@font-face` rules so the fallback font shows immediately.
- **Render-blocking @import in CSS:** replace with separate `<link rel="stylesheet">` tags so the browser can fetch them in parallel.

Best practices

- Use `defer` for scripts that need the DOM: they execute after parsing but before `DOMContentLoaded`.
- Use `async` for fully independent scripts: they execute as soon as they download, in any order.
- **Preload the critical request chain:** tell the browser early about fonts and key assets with `<link rel="preload">`.



- **Audit with Lighthouse:** the Render-blocking resources audit lists every offending file with the estimated time saved by deferring it.

! URL Redirects Test

This URL performed 1 redirects! While redirects are typically not advisable (as they can affect search engine indexing issues and adversely affect site loading time), one redirect may be acceptable, particularly if the URL is redirecting from a non-www version to its www version, or vice-versa.

<https://urbagroupe.ma/events> → <https://urbagroupe.ma/events/>

✓ Largest Contentful Paint Test

The Largest Contentful Paint duration of this webpage is 2.18 seconds. To provide a good user experience, [Google recommends](#) that sites should strive to have Largest Contentful Paint of 2.5 seconds or less.

Largest Contentful Paint element within the viewport:

Text: Créateur d'Événements Inoubliables

Html: `<h1 style="font-size: 5em;">`

✓ Cumulative Layout Shift Test

The CLS score of this webpage is 0.0002. To provide a good user experience, [Google recommends](#) that sites should strive to have a CLS score of 0.1 or less.

DOM element which contributes the most to CLS score:

Text: Accueil Nos Services Nos Références Nos Réalisations Blog Contact

Html: `<ul class="navbar-nav">`

Score: 0.0002



Server and security

3 Failed

0 Warnings

4 Passed

✖ URL Canonicalization Test

<https://urbagroupe.ma/events/> and <https://www.urbagroupe.ma/events/> should resolve to the same URL, but currently do not.

How to pass this test?

This test fails when different URL variants of the site (apex versus www, HTTP versus HTTPS, with or without trailing slash) do not all resolve consistently to a single canonical version. When the same content is reachable at multiple URLs, search engines may split ranking signals across the variants. Fixing this issue means picking one canonical form for the site and 301-redirecting every other variant to it. See the [Google Search Central canonicalization guide](#) for the full specification.

Where to make the change

- **Server configuration:** add 301 redirects in your web server config (Nginx, Apache, IIS) or via `_redirects` on Netlify, `vercel.json` on Vercel, or Cloudflare Page Rules.
- **CDN:** most CDNs offer redirect rules that fire before the origin is hit, which is the cheapest way to enforce canonicalization.
- **Application code:** as a backstop, applications can detect the wrong host or scheme and emit a 301 redirect to the canonical URL.

Common causes and how to resolve them

- **Both apex and www serve the same content:** pick one (most sites prefer apex today) and 301-redirect the other to it permanently.
- **HTTP and HTTPS both serve content:** redirect all HTTP requests to HTTPS, then add HSTS so browsers stop trying HTTP at all.
- **Trailing slash inconsistency:** pick one form (with or without) and redirect the other.
- **Uppercase or mixed-case URLs:** redirect uppercase paths to their lowercase canonical form.
- **Index page accessible at multiple paths:** for example `/index.html`, `/index.php`, and `/` all serving the same content. Redirect everything to `/`.

Best practices

- **Use 301 (permanent) redirects:** they pass the strongest ranking signal and are cached aggressively by browsers.
- **Pair with `<link rel="canonical">`:** the canonical tag is a backup signal for cases the redirect cannot cover (parameterized URLs, syndicated content).
- **Avoid chains:** redirect directly to the final URL in one hop.
- **Update internal links to canonical URLs:** linking to non-canonical variants triggers unnecessary redirects on every visit.





✓ SSL Checker and HTTPS Test

This website is successfully using HTTPS, a secure communication protocol over the Internet.

- ✓ The certificate is not used before the activation date.
- ✓ The certificate has not expired.
- ✓ The hostname "urbagroupe.ma" is correctly listed in the certificate.
- ✓ The certificate should be trusted by all major web browsers.
- ✓ The certificate was not revoked.
- ✓ The certificate was signed with a secure hash.

Certificate Chain:

Server certificate	
Common Name	*.urbagroupe.ma
Subject Alternative Names (SANs)	*.urbagroupe.ma, urbagroupe.ma
Not Valid Before	Sun, May 10th 2026, 4:28:51 am (UTC)
Not Valid After	Sat, August 8th 2026, 4:28:50 am (UTC)
Signature Algorithm	sha256WithRsaEncryption
Issuer	R12
Intermediate certificate	
Common Name	R12
Organization	Let's Encrypt
Location	US
Not Valid Before	Wed, March 13th 2024, 12:00:00 am (UTC)
Not Valid After	Fri, March 12th 2027, 11:59:59 pm (UTC)
Signature Algorithm	sha256WithRsaEncryption
Issuer	ISRG Root X1
Root certificate	
Common Name	ISRG Root X1
Organization	Internet Security Research Group



Location	US
Not Valid Before	Thu, June 4th 2015, 11:04:38 am (UTC)
Not Valid After	Mon, June 4th 2035, 11:04:38 am (UTC)
Signature Algorithm	sha256WithRsaEncryption
Issuer	ISRG Root X1

✓ Mixed Content Test (HTTP over HTTPS)

This webpage does not use mixed content - both the initial HTML and all other resources are loaded over HTTPS.

✓ HTTP2 Test

This webpage is using the HTTP/2 protocol.



✖ Plaintext Emails Test

We've found 1 email addresses in your page code! We advise you [to protect email links](#) in a way that hides them from the spam harvesters.

How to pass this test?

This test fails when the page exposes one or more email addresses in plain text. Spam-harvesting bots scrape pages for plaintext addresses and add them to bulk mailing lists. Fixing this issue means obfuscating contact addresses, using a contact form, or rendering them via JavaScript so simple scrapers cannot read them.

Where to make the change

- **Application code or templates:** replace plain `contact@example.com` text with one of the obfuscation techniques below.
- **WordPress:** a contact form plugin handles email submissions without exposing the address. Some SEO and security plugins also offer email obfuscation.
- **Shopify, Wix, Squarespace:** use the platform's built-in contact form blocks rather than displaying the email address directly.

Common causes and how to resolve them

- **Plain text address in a paragraph or footer:** replace with a contact form, an obfuscated rendering (HTML entities, CSS reversal), or a JavaScript-built mailto link.
- **Mailto link with the address visible:** the visible text is what bots read. Make the visible text a label like "Email us" while the `href` still works.
- **Address baked into images is fine for SEO but bad for accessibility:** screen readers cannot read it. Pair an image with a contact form instead.

Best practices

- **Prefer contact forms:** they reduce spam and make analytics easier.
- **Use HTML entity encoding:** writing `co` for the address defeats the simplest scrapers, though sophisticated ones decode it.
- **Render via JavaScript:** building the mailto link client-side blocks scrapers that only read static HTML.
- **Consider a Cloudflare-style email obfuscation:** some CDNs automatically rewrite plaintext emails to obfuscated equivalents.

✔ Unsafe Cross-Origin Links Test

This webpage is not using `target="_blank"` links without `rel="noopener"` or `rel="noreferrer"` attribute.



Mobile usability

0 Failed

0 Warnings

3 Passed

✓ Meta Viewport Test

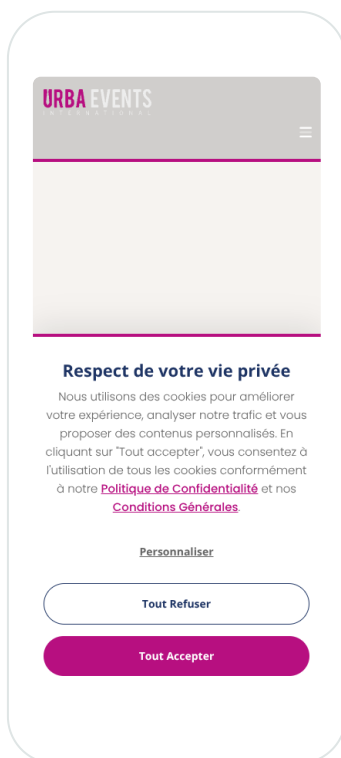
This webpage is using a viewport meta tag.

```
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=5" />
```

✓ Media Query Responsive Test

This webpage is using CSS media queries, which is the base for responsive design functionalities.

i Mobile Snapshot Test





Advanced SEO

2 Failed

1 Warnings

6 Passed



Structured Data Test

This webpage is using structured data.



✖ Custom 404 Error Page Test

This website is not using a custom 404 error page! Default 404 error pages result in a poor experience - it can mislead users into thinking an entire site is down or broken, greatly increases the chance they leave the website entirely, and looks unprofessional. We recommend to have a custom 404 error page in order to improve the website's user experience by letting users know that only a specific page is missing/broken (and not the entire site), providing them helpful links, the opportunity to report bugs, and potentially [track the source of broken links](#).

How to pass this test?

This test fails when the site does not serve a branded, helpful page for missing URLs. A custom 404 keeps users on the site by offering navigation, search, and suggested links rather than dropping them at the browser's generic error screen. Fixing this issue means designing a 404 template, ensuring it returns the correct HTTP 404 status, and pointing your server at it.

Where to make the change

- **Server configuration:** point your web server at a custom 404 page. In Nginx, use `error_page 404 /404.html;`. In Apache, use `ErrorDocument 404 /404.html`.
- **WordPress:** create a `404.php` template in your theme. WordPress invokes it automatically for missing URLs.
- **Shopify:** edit the `404.liquid` template in the theme code editor.
- **Wix or Squarespace:** use the platform's built-in 404 page editor in site settings.
- **Headless or framework sites:** Next.js uses `not-found.js`, Astro uses `404.astro`, and similar frameworks all have a convention for the missing-route page.

Common causes and how to resolve them

- **No custom 404 at all:** create one. The minimum is the site header, navigation, a clear "page not found" message, and links to popular sections.
- **404 page returns HTTP 200:** a "soft 404" confuses search engines. Verify the response status with DevTools or curl, and configure the server to return 404 for missing routes.
- **404 page redirects to homepage:** redirecting users to the homepage hides broken URLs from analytics and search engines. Show a real 404 instead.
- **Generic browser error visible:** the server is not invoking the custom template. Check the configuration directive.

Best practices

- **Keep the site chrome:** show the header, footer, and navigation so users do not feel lost.
- **Offer next steps:** a search box, popular links, or category navigation help users get back on track.
- **Track 404s:** log missing-URL hits in analytics so you can spot broken inbound links worth fixing or redirecting.
- **Be clear about the error:** "Sorry, that page does not exist" is better than vague messaging that confuses users.



✔ Noindex Tag Test

This webpage does not use the noindex meta tag. This means that it can be indexed by search engines.



✖ Canonical Tag Test

This webpage is using the canonical link tag. This tag specifies that the URL: <https://urbaevents.com> should be the preferred version of this page. The canonical tag can be useful when there are similar versions of the same content on several URLs (e.g., such as e-commerce sites where URL modifiers like sort parameters are appended to a product page's URL). Please ensure that this specification is correct, as canonical tags are often hard-coded and may not always reflect the latest changes in a site's URL structure.

```
<link href="https://urbaevents.com" rel="canonical"/>
```



How to pass this test?

The canonical link tag declares the preferred URL for a page when the same or similar content is reachable at multiple URLs. Fixing this issue means adding a `<link rel="canonical">` tag to every page that points at the URL you want indexed, which consolidates ranking signals onto the canonical version and helps AI crawlers identify the source of truth when citing the page. See the [Google canonicalization guide](#) for the full specification.

Example

```
<head>
  <link rel="canonical" href="https://example.com/shoes/marathon">
</head>
```

Where to make the change

- **Raw HTML:** add a single `<link rel="canonical">` inside the `<head>`. The href must be an absolute URL.
- **WordPress:** a dedicated SEO plugin emits canonical tags automatically. Override per-post when the auto-derived URL is wrong.
- **Shopify:** the default Liquid layout emits canonical tags from `{{ canonical_url }}`. Confirm your custom theme has not removed them.
- **Wix or Squarespace:** both platforms emit canonical tags automatically; per-page override fields are available in SEO settings.
- **Headless or framework sites:** emit the canonical tag from the framework's metadata API alongside title and description.

Common causes and how to resolve them

- **No canonical tag at all:** add one pointing at the page's preferred URL.
- **Canonical points to the wrong page:** a templating bug can point every page's canonical at the homepage. Verify the value matches the current URL on a sample of pages.
- **Canonical uses a relative URL:** always use the full `https://` URL. Relative paths can be misinterpreted.
- **Multiple canonical tags on the same page:** only one should exist. A theme and an SEO plugin sometimes both emit one. Pick a single source.
- **Canonical conflicts with a noindex tag:** if a page is noindexed, the canonical pointing at it has no effect. Resolve which signal you actually want.

Best practices

- **Self-canonicalize when in doubt:** if a page has no duplicates, it should canonicalize to itself.
- **Pair with consistent internal linking:** link to the canonical URL from internal navigation, not to query-parameter variants.
- **Use canonicals for pagination and filters:** filtered or sorted product list URLs typically canonicalize to the unfiltered base view.
- **Verify with URL Inspection:** Google Search Console reports the canonical Google has chosen, which may differ from the one you declared.



Nofollow Tag Test

This webpage does not use the nofollow meta tag. This means that search engines will crawl all links from this webpage.

Disallow Directive Test

This website lacks a "robots.txt" file. This file can protect private content from appearing online, save bandwidth, and lower load on your server. A missing "robots.txt" file also generates additional errors in your apache log whenever robots request one.

Meta Refresh Test

This webpage is not using a meta refresh tag.

SPF Records Test

This DNS server is using an SPF record.

```
v=spf1 ip4:176.9.116.24 +a +mx +ip4:157.90.129.108 ~all
```

Ads.txt Validation Test

This website doesn't use an ads.txt file! Ads.txt is a text file that contains a list of Authorized Digital Sellers. The purpose of ads.txt file is to give advertisers and advertising networks the ability to verify who is allowed to sell